

Begonnen am	Freitag, 15. November 2024, 23:33
Status	Beendet
Beendet am	Freitag, 24. Januar 2025, 11:33
Verbrauchte Zeit	69 Tage 12 Stunden

Information

Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

Frage 1

Richtig

Nicht bewertet

Data Transfer Instructions

Please classify the given ARM Cortex M0 Data Transfer instructions. Note, that some variants are not legal instructions.

Instruction	Classification
MOV R3,R9	Register-to-Register
STR R1,[R10]	Illegal-Instruction
LDR R6,[PC,#32]	Loading-Literals
MOVS R3,R9	Illegal-Instruction
LDR R1,[R1]	Loading-Data
MOVS R6,=0x12	Illegal-Instruction
LDR R6,label	Loading-Literals
MOVS R7,R1	Register-to-Register
STR R1,[R2]	Storing-Data
LDR R6,=value	Loading-Literals
MOVS R6,#0x1234	Illegal-Instruction
MOVS R6,#0x12	Loading-Literals
LDRH R4,=0x1234	Illegal-Instruction

- MOV Rx,Ry: low and high registers
- MOVS Rx,Ry: low registers only
- MOVS Rx,#item: 1-byte value only
- LDRx Rx,[Ry,Rz]: low registers only
- LDRx Rx,[Ry,#offset]: low registers only
- LDR Rx,label: PC relative, loads 4 bytes from the location indicated by the label
- LDR Rx,=item: convenience pseudo instruction boiling down to LDR Rx,label behind the scene - the item is placed at the discretion of the compiler, the location given some unique-anonymous-label, and finally accessed by the LDR Rx,unique-anonymous-label
- STRx Rx,[Ry,Rz]: low registers only
- STRx Rx,[Ry,#offset]: low registers only
- The only place where = precedes a label or value is to indicate the LDR pseudo instruction - all other references to direct values are preceded by #.
- LDRx and STRx are the only instructions that have address expressions in [...].

Data Transfer Tasks - Advanced

Write the ARM Cortex M0 assembly instructions that complete the given tasks.

Notes:

- You are expected to give each answer with at most two instructions - set an empty field to a single semi-colon ';'.
- The automated check of the answers expect no redundant spaces.
- Use R7 for intermediate store if needed.

a) Copy contents of R1 to R3 so that **no flags are affected.**

#	Instruction
1.	MOV R3,R1
2.	;

b) Initialize R0 with value 0xAA so that **no flags are affected.**

#	Instruction
1.	LDR R0,#0xAA
2.	;

c) Initialize R1 with value 234 so that **flags are affected.**

#	Instruction
1.	MOVS R1,#234
2.	;

d) Initialize R4 with value 0xAABBCC so that **no flags are affected.**

#	Instruction
1.	LDR R4,#0xAABBCC
2.	;

e) Copy contents of R9 to R3 so that **no flags are affected.**

#	Instruction
1.	MOV R3,R9
2.	;

f) Initialize R10 with value 0x345678 so that **no flags are affected.**

#	Instruction
1.	LDR R7,#0x345678
2.	MOV R10,R7

- copy values between registers is done by some MOV(S) instruction even the value remains in the original register (i.e. not moved but copied)
- MOV**S** modifies N and Z flag, depending of the result value (i.e. depending on the value that is set in the destination register) - all other MOV/LDR/STR instructions do not affect the flags
- loading a literal value (e.g. MOVS R1,#123, or LDR R1,#123) into a register always sets all 32 bits of the destination register
- loading a literal value by MOVS only supports single byte values as argument, setting the register's content to the lower byte to the given value and the upper 3 bytes to zero
- there is no **MOV (without S)** instruction with a **literal value** as argument - therefore, if loading a value into a register **without affecting the flags**, the **LDR, Rx,=value** has to be used
- you can use any supported base to express literal values (i.e. decimal or hex) - the resulting value is the respective bit pattern from the given number - e.g. 0x68 = 104 = binary
0000'0000'0000'0000'0000'0000'0110'1000
- if high registers are involved, the only instruction is MOV which supports these - if at the same time, the N and Z flags have to be set accordingly, some MOVS instruction has to be involved to the expense of an additional low register to be used, e.g. MOV R10,R9, MOV R1,R10, MOVS R1,R1

Frage 3

Richtig

Nicht bewertet

Data Transfer with High Registers

Which instructions can be used in connection with register **R10**?

- ☐ LDR
- ☐ LDRB
- ☐ LDRH
- ☐ STR
- ☐ STRB
- ☐ STRH
- ☒ MOV
- ☐ MOVS

Erreichte Punkte 1,00 von 1,00

Die richtige Antwort ist:

- MOV

R10 is a high register. The only one of the given instructions is MOV (without trailing S) that takes low **and** high registers.

Frage 4

Richtig

Nicht bewertet

Data Transfer: C-code array versus assembly instructions

For the given C code below, choose the most appropriate ARM Cortex M0 data transfer instructions.

```
100 // R0 holds the address of array
101 // R1 holds the value of variable max_value
102 // R2 holds the value of variable current_value
103 // R3 holds the value of variable i
104 uint8_t max_value = 0;
105 if (size > 0) {
106     max_value = array[0];
107     for (size_t i = 1; i < size; ++i) {
108         uint8_t current_value = array[i];
109         if (current_value > max_value) {
110             max_value = current_value;
111         }
112     }
113 }
```

Note that most of the given instructions are of wrong syntax, so, the task is to first filter out the syntactically wrong instructions and then choose between the remaining ones.

Most appropriate ARM Cortex M0 data transfer instruction for **line 100**:

Most appropriate ARM Cortex M0 data transfer instruction for **line 106**:

Most appropriate ARM Cortex M0 data transfer instruction for **line 108**:

- Line 100:
 - MOV R0,array: **wrong syntax**: MOV only between registers
 - MOV R0,#array: **wrong syntax**: MOV only between registers
 - MOV R0,=array: **wrong syntax**: MOV only between registers
 - MOVS R0,array: **wrong syntax**: missing # before literal argument
 - MOVS R0,#array: correct syntax but wrong logic: the *array* label is most likely not of 1.byte size
 - MOVS R0,=array: **wrong syntax**: = is not allowed here
 - LDR R0,array: correct syntax but wrong logic: this would load the value located at label *array* and not the address the label *array* stand for
 - LDR R0,#array: **wrong syntax**: # is not allowed here
 - LDR R0,=array: correct syntax and correct logic: load the address the label *array* stand for
- Line 106:
 - LDR R1,[R0,#0]: correct syntax but wrong logic: LDR loads **4 bytes** from the given address while the statement at line 106 indicates to only load **one byte**
 - LDR R1,[R0]: correct syntax but wrong logic: LDR loads **4 bytes** from the given address while the statement at line 106 indicates to only load **one byte**
 - LDRB R1,[R0,#0]: correct syntax and correct logic: loads **one byte** from address given by **R0 + 0**
 - LDRB R1,[R0]: correct syntax and correct logic: same instruction as LDRB R1,[R0,#0] - the **omitted offset** is **assumed** to be **zero**
 - LDRB R1,[array]: **wrong syntax**: the [...] expression must be one of [Rx,Ry] or [Rx,#offset] or [Rx]
 - LDRB R1,R0: **wrong syntax**: the second argument part must be an address expression of the form [...]
- Line 108:
 - LDR R2,[R0,R3]: correct syntax but wrong logic: LDR loads **4 bytes** from the given address while the statement at line 108 indicates to only load **one byte**
 - LDRB R2,[R0,R3]: correct syntax and correct logic: loads **one byte** from address given by **R0 + R3**
 - LDRB R2,[array+R3]: **wrong syntax**: neither

- supported to give a **label** instead of a register and **nor** supported to do an **addition** in the [...] address expression
- LDRB R2,[array,R3]: **wrong syntax: not** supported to give a **label** instead of a register in the [...] address expression
- LDRB R2,array[R3]: **wrong syntax:** even it looks tempting coming from C code, this is not proper assembly syntax - the second argument part must be an address expression of the form [...]
- LDRB R2,array,R3: **wrong syntax:** the second argument part must be an address expression of the form [...]

Frage 5

Richtig

Nicht bewertet

Data Transfer - Memory and Alignment

Please indicate for the given code fragment how the compiler allocates memory for constants and global variables.

Note: Depending on the directives, some padding bytes have to be inserted to respect the alignment constraints.

```

limit      AREA constants, DATA, READONLY
DCD        5
text       DCB    "Hello World"
           DCB    0
text_addr  DCD    text

           AREA variables, DATA, READWRITE
counter    DCD    10
buffer     SPACE 10
colors     DCW    0xFFFF, 0xFF80, 0xFF00, 0xFE80
last_counter DCD    200

```

The constants start at 0x08000200, the variables start at 0x20000000.

Give the byte values as

- hex values (with or without leading **0x..**)
- unknown values to be indicated by a question mark **"?"**
- padding bytes to respect alignment constraint of the following item with **"pad"**

variables

constants

Address	Byte Value
0x08000200	0x05
0x08000201	0x00
0x08000202	0x00

Address	Byte Value
0x20000000	0x0A
0x20000001	0x00
0x20000002	0x00
0x20000003	0x00

0x08000203	0x00	0x20000004	0x00
0x08000204	0x48	0x20000005	0x00
0x08000205	0x65	0x20000006	0x00
0x08000206	0x6c	0x20000007	0x00
0x08000207	0x6c	0x20000008	0x00
0x08000208	0x6f	0x20000009	0x00
0x08000209	0x20	0x2000000A	0x00
0x0800020A	0x57	0x2000000B	0x00
0x0800020B	0x6F	0x2000000C	0x00
0x0800020C	0x72	0x2000000D	0x00
0x0800020D	0x6C	0x2000000E	0xFF
0x0800020E	0x64	0x2000000F	0xFF
0x0800020F	0x00	0x20000010	0x80
0x08000210	0x04	0x20000011	0xFF
0x08000211	0x02	0x20000012	0x00
0x08000212	0x00	0x20000013	0xFF
0x08000213	0x08	0x20000014	0x80
0x08000214	?	0x20000015	0xFE
0x08000215	?	0x20000016	pad
0x08000216	?	0x20000017	pad
0x08000217	?	0x20000018	0xC8
0x08000218	?		

0x08000219	?	0x20000019	0x00
0x0800021A	?		
0x0800021B	?	0x2000001A	0x00
		0x2000001B	0x00

- limit: 0x08000200, 1x 4-bytes (DCD), no leading padding needed since 0x08000200 already aligned (divisible by 4 due to DCD), value = 0x00000005 (Little Endian)
- text: 0x08000204, 11x 1-byte (DCB), no leading padding needed since 1-byte values, values = ASCII code per character (no implicit trailing zero as in C!)
- 0x0800020F, 1x 1-byte (DCB), no leading padding needed since 1-byte value, value = 0x00
- text_addr: 0x08000210, 1x 4-byte (DCD), no leading padding needed since 0x08000210 already aligned (divisible by 4 due to DCD), value = the address the text label stands for = 0x08000204 (Little Endian)
- counter: 0x20000000, 1x 4-byte (DCD), no leading padding needed since 0x20000000 already aligned (divisible by 4 due to DCD), value = 0x0000000A (Little Endian)
- buffer: 0x20000004, 10x 1-byte (SPACE 10), no leading padding needed since 1-byte values, values = all 0x00
- colors: 0x2000000E, 4x 2-byte (DCW), no leading padding needed since 0x2000000E already aligned (divisible by 2 due to DCW), values = 0xFFFF, 0xFF80, 0xFF00, 0xFE80 (Little Endian)
- last_counter: 0x20000018, 1x 4-byte (DCD), **leading padding needed** (2x 1-byte 0x00) since 0x20000016 is **not aligned** (not divisible by 4 due to DCD), value = 0x000000C8 (Little Endian)

Frage 6

Richtig

Nicht bewertet

Data Transfer: Assembly Text to Machine Code

The assembly instructions are translated by the compiler into ARM Cortex M0 machine code.

In this exercise, you do that translation yourself.

Take the Handouts from Moodle (section *ARM v6-M Instruction Set*) to look up the respective bit patterns.

ARM Cortex M0 Assembly Instruction	Bit Pattern			Resulting Code in Hex
	Instruction Bits	Register Bits	Immediate Value Bits	
MOVS R3,#123	00100	011	01111011	237B

ARM Cortex M0 Assembly Instruction	Bit Pattern				Resulting Code in Hex
	Instruction Bits	Register Bits	Register Bits	Register Bits	
STR R1, [R2,R3]	0101000	011	010	001	50D1

ARM Cortex M0 Assembly Instruction	Bit Pattern			Resulting Code in Hex
	Instruction Bits	Register Bits	Offset Bits	
LDR R5, [PC,#0x4C]	01001	101	00010011	4D13

- **MOVS R3,#123** : 00100 011 01111011, decimal 123 = hex 0x7B as immediate value.
- **STR R1,[R2,R3]** : 0101 000 011 010 001, note that the sequence of the registers in the assembly is inverse to the sequence in the machine code.
- **LDR R5,[PC,#0x4C]** : 01001 101 00010011, note that the assembly offset is right shifted by 2 for encoding in the machine code since the address must be 4-byte aligned, thus, the lower two bits of the assembly text offset must be zero and these two zero bits are not stored in the machine code.

Frage 7

Richtig

Nicht bewertet

Data Transfer: Machine Code to Assembly Text

In this exercise, you disassemble ARM Cortex M0 machine code.

Take the Handouts from Moodle (section *ARM® Thumb® Cortex-M0/M1 Instruction Set*) to look up the respective bit patterns.

Machine Code	Bit Pattern Split Into Groups	ARM Cortex M0 Assembly Instruction
0x81EB	10000 00111 101 011	STRH R3,[R5,#0x0E]

Machine Code	Bit Pattern Split Into Groups	ARM Cortex M0 Assembly Instruction
0x4F2B	01001 111 00101011	LDR R7,[PC,#0xAC]

Machine Code	Bit Pattern Split Into Groups	ARM Cortex M0 Assembly Instruction
0x468C	01000110 1 0001 100	MOV R12,R1

- 0x81EB = 10000 00111 101 011 = **STRH R3**, **[R5,#0x0E]**, note that the encoded offset is **left shifted by one bit** to get an assembly text offset that divides by 2 since STRH must access a **halfword aligned address**.
- 0x4F2B = 01001 111 00101011 = **LDR R7**, **[PC,#0xAC]**, note that the encoded offset is **left shifted by two bits** to get an assembly text offset that divides by 4 since LDR must access a **word aligned address**.
- 0x468C = 01000110 1 0001 100 = **MOV R12,R1**, note that the sequence of the registers in the encoding and in the assembly text is inverse, and that the destination register encoding is split in the machine code (the most significant bit is at bit7, the remaining three bits at bit2..0).

Frage 8

Vollständig

Nicht bewertet

Data Transfer: Literal Pool

Enter below some ARM Cortex M0 assembly text with the given constraints:

1. Assembly text to load 0xAABBCCDD into R7 by using the pseudo instruction LDR Rx,=... (i.e., by employing the literal pool machinery).
2. Assembly text with identical effect, but not using the pseudo instruction LDR Rx,=... (i.e., by not employing the literal pool machinery).

```
LDR R7,=0xAABBCCDD
LDR R7,label

label DCD 0xAABBCCDD
```

1. With literal pool: **LDR R7,=0xAABBCCDD**

Note that the value 0xAABBCCDD is implicitly placed as a DCD value at a convenient location (this is the **literal pool**) in the program code and then accessed by **LDR R7,[PC,#offset-to-the-placed-value]**.

2. Without literal pool:

LDR R7,some_label

...

some_label DCD 0xAABBCCDD

Notes:

- The **LDR R7,some_label** has **no leading =** before the label.
- This instruction (**without leading =**) translates into **LDR R7,[PC,#offset-to-some-label]** where the compiler calculates the respective offset for you.

Frage 9

Richtig

Nicht bewertet


 Barrierefreiheitserk
 Barrierefreiheit

Data Transfer: PC relative offsets - Advanced

The compiler provides the very powerful convenience concept of labels. With the labels, you gain flexibility to refer to other code locations in an abstract way.

Lets play now compiler and calculate the PC relative offset manually - and appreciate the benefit by letting the compiler doing so 😊

Address	Assembly Text
0x08000200	LDR R5,[PC, #0x7C] ; value is 0x80 bytes away
0x08000202	LDR R6,[PC, #0x7C] ; value is 0x7E bytes away
0x08000204	LDR R7,[PC, #0x78] ; value is 0x7C bytes away
...	
0x08000280	DCD 0x12345678

Note: The convenient code would be to set a **label at address 0x08000280** and to load the value 0x12345678 at that label into the register by **LDR R_x,label** (which the compiler translates into some **LDR R_x,[PC,#some-offset]**).

- 0x08000200: add 4 to this address and round down to the 4-byte aligned address (=0x08000204), then calculate the difference to the target address (0x08000280 - 0x08000204 = **0x7C**).
- 0x08000202: add 4 to this address and round down to the 4-byte aligned address (=0x08000204), then calculate the difference to the target address (0x08000280 - 0x08000204 = **0x7C**).
- 0x08000204: add 4 to this address and round down to the 4-byte aligned address (=0x08000208), then calculate the difference to the target address (0x08000280 - 0x08000208 = **0x78**).

Note that there are various approaches to describe the address calculation - the effect is always the same

- The lectures state that the PC points to the **next** instruction's address (i.e. current instruction's PC plus 2) and from that PC to **round up** the address to the **next address** that is divisible by 4.
- The same result is achieved by **adding** to the **current** instruction's PC the value **4** and **rounding down** to the address that is divisible by 4.

Frage 10

Nicht

beantwortet

Nicht bewertet

Data Transfer: C pointer versus assembly text

Implement the given C code in ARM Cortex M0 assembly.

```
char s[10];
int i;
int* p;

...
s[3] = 'A';
...
p = &i;
...
*p = 19;
```

```
s SPACE 10      ; or DCB 0,0,0,0,0,0,0,0,0,0
i SPACE 4       ; or DCD 0
p SPACE 4       ; or DCD 0

...
MOVW R6, #0xA   ; the value to store in s[3]
LDR  R7, =s      ; variables in C are an abstraction for the location of the memory that holds the respective value
STWB R6, [R7, #3] ; store the value in *(s+3), i.e. in s[3]
...
LDR  R7, =p      ; variables in C are an abstraction for the location of the memory that holds the respective value
LDR  R6, =1       ; variables in C are an abstraction for the location of the memory that holds the respective value
STR  R6, [R7]     ; store the i (i.e. its address) in variable p (at the address of p)
...
MOVW R5, #19     ; the value to store in i over *p
LDR  R6, [R7]    ; get the location of the object i via the variable p
STR  R5, [R6]     ; store the value at i
```

[◀ Lab - Introduction to Assembly](#)[Direkt zu:](#)[Lecture Arithmetic Operations ▶](#)